

1.5 Database Models

A **Database Model** is a logical way of organizing, storing, and relating data in a database.

The three major database models are:

1. **Hierarchical Database Model**
 2. **Network Database Model**
 3. **Relational Database Model**
-

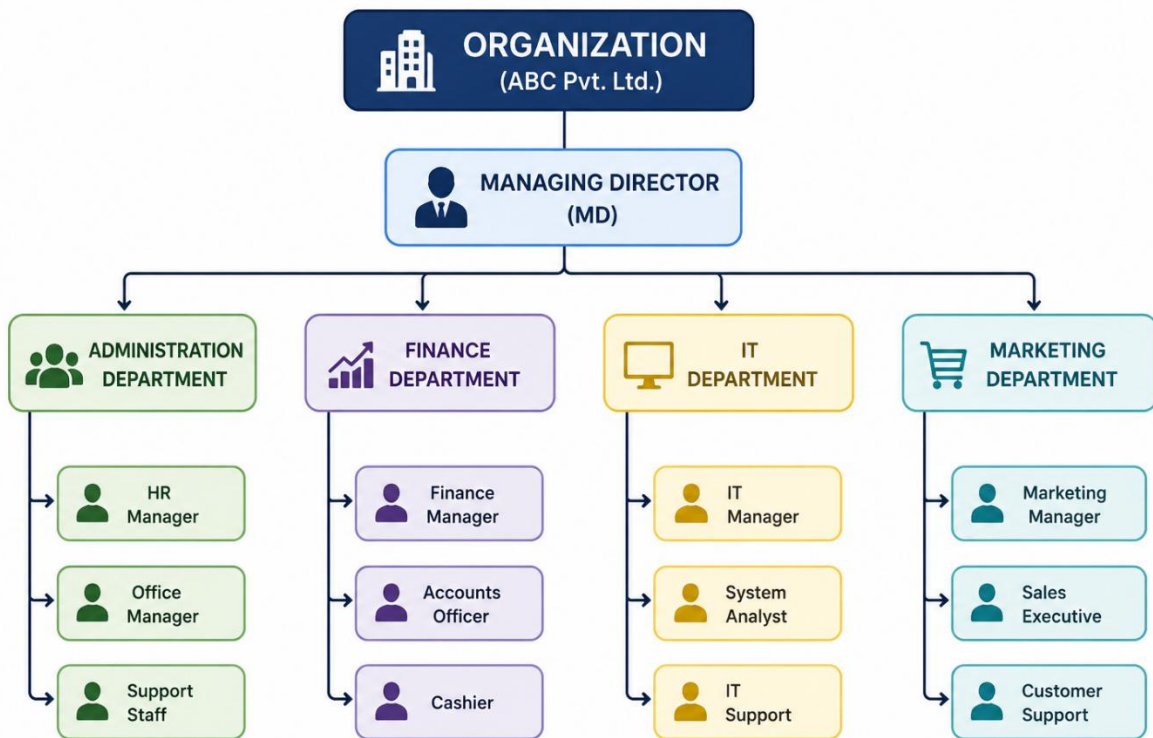
1. Hierarchical Database Model

A. English Explanation

Definition

1. The **Hierarchical Database Model** organizes data in a **tree-like structure**.
2. Data is arranged using **parent-child relationships**.
3. The topmost record is called the **root**.
4. A parent can have multiple children.
5. Normally, a child has only **one parent**.
6. It mainly represents **one-to-many (1:M)** relationships.

Structure



Important Points

- Uses a **tree structure**.
- Has a **root** at the top.
- Uses **parent-child relationships**.
- Data is generally accessed from **top to bottom**.
- Suitable for data that naturally follows a hierarchy.
- Mainly supports **1:M relationships**.
- It is simple and easy to understand.

Example

Consider a school database:

School
↓
Class

↓
Student

Here:

- School = Parent
- Class = Child
- Student = Further child

Advantages

1. Simple structure.
2. Easy to understand.
3. Fast data access when the hierarchy is known.
4. Clearly represents parent-child relationships.
5. Suitable for hierarchical data.

Disadvantages

1. Difficult to represent many-to-many relationships.
2. Data duplication may occur.
3. Less flexible.
4. Structural changes can be difficult.
5. Not suitable for complex relationships.

Examples

- Organization structure
- File and folder systems
- Company → Department → Employee
- School → Class → Student

Memory Trick

Hierarchical = TREE = Parent → Child

2. Network Database Model

A. English Explanation

Definition

1. The **Network Database Model** organizes data in a **network or graph-like structure**.
2. Records are connected through **links or pointers**.
3. A record can have **multiple relationships** with other records.
4. Unlike the hierarchical model, a child can have **multiple parents**.
5. It supports **one-to-one, one-to-many, and many-to-many relationships**.
6. It is more flexible than the hierarchical model.

Structure

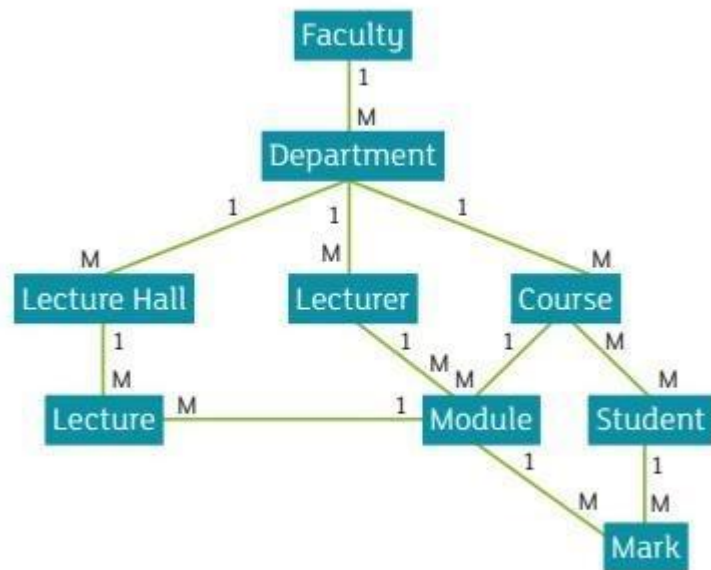
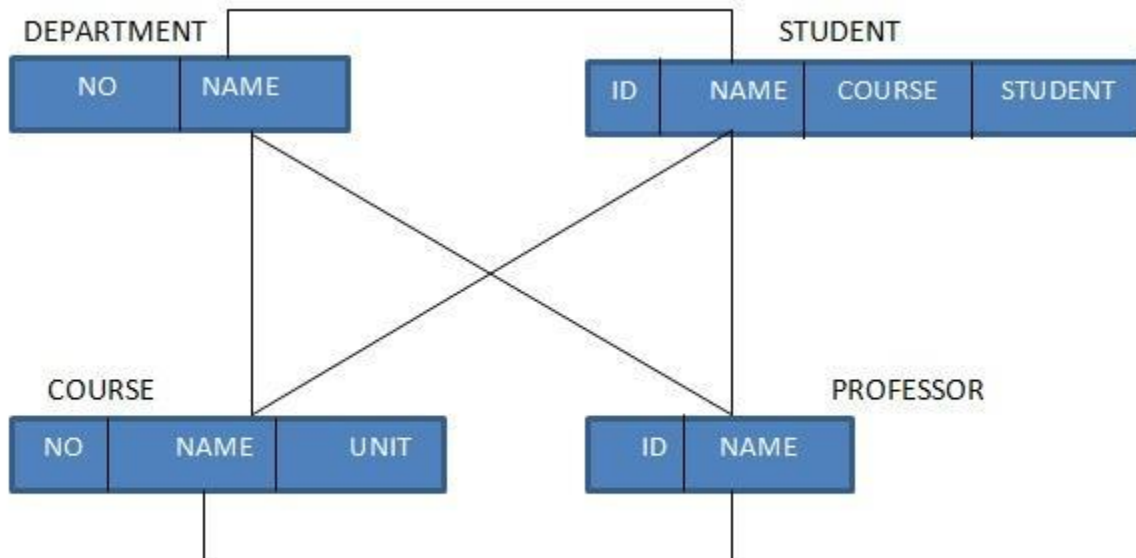


Figure 5.3 – Network Database Model

OR,



Important Points

- Uses a **network/graph structure**.
- Records are connected using **links**.
- A record may have multiple parents.
- Supports **1:1 relationships**.
- Supports **1:M relationships**.
- Supports **M:N relationships**.
- More flexible than the hierarchical model.
- Data can be accessed through different paths.

Example

A college database:

Student ↔ Subject

A student can study many subjects.

A subject can be studied by many students.

Therefore:

Student ↔ Subject = Many-to-Many relationship

Advantages

1. Supports complex relationships.
2. Supports many-to-many relationships.
3. More flexible than the hierarchical model.
4. Multiple paths can be used to access data.
5. Data retrieval can be fast through predefined links.

Disadvantages

1. Complex to design.
2. Difficult to understand compared to hierarchical models.

3. Difficult to maintain.
4. Structural changes can be difficult.
5. Users need to understand the relationships and links.

Examples

- Student ↔ Subject
- Customer ↔ Product
- Employee ↔ Project

Memory Trick

Network = WEB = Multiple Connections

3. Relational Database Model

A. English Explanation

Definition

1. The **Relational Database Model** organizes data into **tables**.
2. A table consists of **rows and columns**.
3. A row represents a **record or tuple**.
4. A column represents a **field or attribute**.
5. Tables are related using **keys**.
6. **Primary Key** uniquely identifies a record.
7. **Foreign Key** establishes a relationship between tables.
8. SQL is commonly used to create, manipulate, and retrieve data.

9. It is the **most widely used database model**.

Example

STUDENT

Student_ID	Name	Class
101	Ram	12
102	Sita	12
103	Hari	12

Here:

- STUDENT = Table
- Student_ID, Name, Class = Columns
- 101, Ram, 12 = Row/Record
- Student_ID = Primary Key

Primary Key

1. A Primary Key uniquely identifies each record.
2. It cannot contain duplicate values.
3. It normally cannot contain NULL values.

Example:

Student_ID → Primary Key

Foreign Key

1. A Foreign Key is used to establish a relationship between tables.
2. It refers to the Primary Key of another table.

```
STUDENT
Student_ID (PK)
    ↓
    ↓
ENROLLMENT
Student_ID (FK)
```

Important Points

- Data is stored in **tables**.
- Tables contain **rows and columns**.
- Rows are called **records/tuples**.
- Columns are called **fields/attributes**.
- Uses **Primary Key and Foreign Key**.
- Supports relationships between tables.
- Uses **SQL**.
- Data redundancy can be reduced through **normalization**.
- Provides good data integrity.
- Highly flexible and widely used.

Advantages

1. Simple and easy-to-understand structure.
2. Data is organized into tables.
3. SQL makes data retrieval easy.
4. Supports relationships between tables.
5. Reduces data redundancy through normalization.
6. Maintains data integrity.
7. Easy to insert, update, delete, and retrieve data.
8. Suitable for large databases.

9. Widely supported by DBMS software.

Disadvantages

1. Complex queries may take more processing time.
2. Managing many related tables can become complex.
3. Large databases may require significant storage and resources.
4. Not ideal for every type of unstructured or graph-based data.

Examples

- MySQL
- Oracle Database
- PostgreSQL
- Microsoft SQL Server
- MariaDB

Memory Trick

Relational = TABLE = Rows + Columns + Keys

B. Nepali Explanation

१. Hierarchical Database Model

परिभाषा

1. Hierarchical model मा data लाई **tree structure** मा organize गरिन्छ।
2. यसमा **parent-child relationship** हुन्छ।
3. सबैभन्दा माथिको record लाई **root** भनिन्छ।
4. एउटा parent का धेरै children हुन सक्छन्।
5. सामान्यतया एउटा child को एउटा मात्र **parent** हुन्छ।
6. यसले मुख्यतः **one-to-many (1:M)** relationship represent गर्छ।

उदाहरण

विद्यालय

↓

कक्षा

↓

विद्यार्थी

- विद्यालय = Parent
- कक्षा = Child
- विद्यार्थी = Further Child

फाइदा

1. Structure simple हुन्छ।
2. बुझ्न सजिलो हुन्छ।

3. Data retrieval छिटो हुन सकछ।
4. Parent-child relationship स्पष्ट हुन्छ।
5. Hierarchical data का लागि उपयुक्त हुन्छ।

बेफाइदा

1. Many-to-many relationship represent गर्न गाह्रो हुन्छ।
2. Data duplication हुन सकछ।
3. Flexibility कम हुन्छ।
4. Structure परिवर्तन गर्न गाह्रो हुन्छ।
5. Complex relationship का लागि suitable हुँदैन।

समझने तरिका

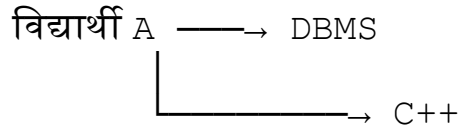
Hierarchical = Tree = Parent → Child

२. Network Database Model

परिभाषा

1. Network model मा data लाई **network/graph structure** मा organize गरिन्छ।
2. Records लाई **links/pointers** द्वारा connect गरिन्छ।
3. एउटा record का धेरै relationships हुन सकछन्।
4. एउटा child का **multiple parents** हुन सकछन्।
5. यसले **1:1, 1:M र M:N** relationships support गर्छ।
6. यो hierarchical model भन्दा बढी flexible हुन्छ।

उदाहरण



विद्यार्थी B $\longrightarrow$ DBMS

यहाँ धेरै students ले एउटै DBMS subject पढ्न सक्छन्।

त्यसैले:

Student $\leftrightarrow$ Subject = Many-to-Many

फाइदा

1. Complex relationships support गर्छ।
2. Many-to-many relationship support गर्छ।
3. Hierarchical भन्दा flexible हुन्छ।
4. Multiple paths बाट data access गर्न सकिन्छ।
5. Predefined links का कारण data retrieval fast हुन सक्छ।

बेफाइदा

1. Design गर्न complex हुन्छ।
2. बुझ्न गाह्रो हुन्छ।
3. Maintain गर्न गाह्रो हुन्छ।
4. Structure परिवर्तन गर्न कठिन हुन सक्छ।
5. Links र relationships बुझ्न आवश्यक हुन्छ।

समझने तरिका

Network = Web = धेरै Connections

३. Relational Database Model

परिभाषा

1. Relational model मा data लाई **tables** मा store गरिन्छ।
2. Table मा **rows** र **columns** हुन्छन्।
3. Row लाई **record/tuple** भनिन्छ।
4. Column लाई **field/attribute** भनिन्छ।
5. Tables बीच relationship बनाउन **keys** प्रयोग गरिन्छ।
6. Primary Key ले record लाई uniquely identify गर्छ।
7. Foreign Key ले tables बीच relationship establish गर्छ।
8. SQL प्रयोग गरेर data manage गर्न सकिन्छ।
9. यो सबैभन्दा **widely used database model** हो।

उदाहरण

STUDENT

Student_ID	Name	Class
101	Ram	12
102	Sita	12
103	Hari	12

- STUDENT = Table
- Student_ID, Name, Class = Columns

- प्रत्येक horizontal entry = Record
- Student_ID = Primary Key

फाइदा

1. Structure simple हुन्छ।
2. बुझ्न सजिलो हुन्छ।
3. Data tables मा व्यवस्थित हुन्छ।
4. SQL प्रयोग गर्न सकिन्छ।
5. Tables बीच relationship बनाउन सकिन्छ।
6. Data redundancy कम गर्न सकिन्छ।
7. Data integrity maintain गर्न सकिन्छ।
8. Data insert/update/delete/retrieve गर्न सजिलो हुन्छ।
9. Large database का लागि suitable हुन्छ।

बेफाइदा

1. Complex queries ले बढी processing time लिन सक्छ।
2. धेरै related tables हुँदा database complex हुन सक्छ।
3. Large database मा धेरै resources आवश्यक पर्न सक्छ।
4. सबै प्रकारका unstructured data का लागि suitable हुँदैन।

समझने तरिका

Relational = Table = Rows + Columns + Keys



Final Revision Table

Point	Hierarchical	Network	Relational
Structure	Tree 	Graph/Web 	Table 
Main concept	Parent–Child	Multiple Links	Rows–Columns
Relationship	Mainly 1:M	1:1, 1:M, M:N	Through Keys
Multiple parents	No	Yes	Yes
M:N	Difficult	Yes	Yes
Flexibility	Low	High	High
Access	Hierarchical navigation	Links/Pointers	SQL
Main key concept	Parent/Child	Links	PK/FK
Common usage	Limited	Limited	★ Very common

Super-short NEB Revision

Hierarchical → Tree → Parent/Child

Network → Graph → Multiple Connections

Relational → Table → Rows/Columns → PK/FK